Department of Computer Science and Engineering

Course Code & Name	CSA0620 – Design and Analysis of Algorithms
Assessment	CO2 – Debugging and Optimisation Task
Course Outcome Assessed	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems.
Maximum Marks	**50**
Duration	1.5 hrs (Assessment conducted in class)
Mode	Individual
Student Reg. No.	**192521093**
Student Name	**DHARSHITHA.K**

Instructions: Debug the code by reasoning and testing, not by rewriting it from scratch. For every change, state what was wrong, why it was wrong and how you verified the fix. Optimisations must preserve the corrected outputs. Marks for each part are shown in brackets.

DEBUGGING AND OPTIMISATION TASK (50 MARKS)

Scenario – SOC Alert-Triage Module of "SentinelOps": SentinelOps is the Security Operations Centre (SOC) of a cloud-services company. A junior developer wrote the Python module below to (1) sort incoming alerts by severity, highest first, keeping alerts of equal severity in arrival order; (2) check whether a source IP address is on the blocklist; (3) find every position of an attack signature in a log line; and (4) find the two login attempts whose locations are closest to each other, which the SOC uses to flag clustered credential-stuffing attacks. The module passed code review but failed in production: alerts disappeared or were duplicated, blocked IP addresses got through, signatures at the end of a log were missed while false signatures were reported, and the "closest logins" report flagged logins hundreds of kilometres apart. The module must be fixed before it processes about 50,000 alerts and batches of 8,000 login events per day.

Code under investigation:

```
import math

def sort_alerts(alerts):          # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts

def is_blocked(blocklist, ip):      # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False

def find_signature(log, pattern):   # brute-force string matching, all
occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        if j == m - 1:
            hits.append(i)
    return hits

def closest_logins(events):         # brute-force closest pair of login locations
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

Test data:

```
alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries   = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events    = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]   # (x, y) in km
```

(a) Reproduce and Identify the Defects (10 Marks): Run the module on the test data. For each function, record the expected output and the actual output, then locate every defect: state the function, the faulty line and the nature of the error. The module contains nine defects, and none of them is marked in the code.

ANSWER:

• Expected outputs:

- Sorted alerts: [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)].
- Blocklist queries: 10.0.0.9 → True; 10.0.0.5 → True; 8.8.8.8 → False.
- Signature positions in "FAILFAIXFAIL": [0, 8].
- Closest login pair: events at indices (0, 5), distance $\sqrt{2} \approx 1.414$ km? No. The closest pair is (0, 5): coordinates (2,3) and (3,4), distance $\sqrt{2} \approx 1.414$ km.

• Actual outputs from the supplied faulty code:

- Sorting does not correctly move the first element and can overwrite an alert.
- Blocklist function returns False immediately, even when the IP is present.
- Signature scan misses the final valid position and can report a partial match.
- Closest-pair function selects the farthest pair rather than the nearest pair, and includes self-pairs in its checks.

• Nine defects:

1. sort alerts: condition j > 0 should be j >= 0; the first element is not checked.
2. sort alerts: final assignment alerts[j] = key should be alerts[j + 1] = key; otherwise an element can be overwritten.
3. is blocked: return False on a match should be return True.
4. is blocked: the else return is inside the loop; move return False after the loop.
5. find signature: range(n - m) should be range(n - m + 1); the last possible starting position is skipped.
6. find signature: j == m - 1 is not a valid full-match test; use a matched flag or Python's for-else pattern.
7. closest logins: min_dist = 0 should be min_dist = float('inf').
8. closest logins: j should start from i + 1, not 0; this removes self-pairs and repeated pairs.

9. closest logins: d > min_dist should be d < min_dist; the task asks for the minimum distance.

(b) Explain the Root Causes and Debug Systematically (12 Marks): For each defect, explain its root cause and how it produces the observed wrong output. Correct the functions one at a time, supporting your reasoning with a step-by-step trace (for example, the insertion trace for the alerts list and the alignment table for the log), and re-test after every correction. Include boundary tests such as an empty blocklist, a pattern at the very end of the log, a pattern longer than the log, equal severities and fewer than two login events.

ANSWER:

• Defects 1 and 2 – sort_alerts:

- j > 0 stops the shifting loop before index 0, so the first alert may remain out of order.
- alerts[j] = key writes the key into the wrong position and can overwrite an item.
- Fix: use j >= 0 and assign alerts[j + 1] = key.
- Stable sorting is preserved by shifting only when the previous severity is strictly smaller. Equal-severity alerts keep their arrival order.

• Insertion-sort trace for the given alerts:

- Start: A1(3), A2(9), A3(5), A4(9), A5(1)
- Insert A2: A2(9), A1(3)
- Insert A3: A2(9), A3(5), A1(3)
- Insert A4: A2(9), A4(9), A3(5), A1(3)
- Insert A5: A2(9), A4(9), A3(5), A1(3), A5(1)
- Final: A2, A4, A3, A1, A5.

• Defects 3 and 4 – is_blocked:

- A matching IP must return True.
- Returning False inside the loop stops the search after checking only the first entry.
- Fix: return True when a match is found; return False only after the loop finishes.
- Boundary test: an empty blocklist returns False. A present IP returns True; an absent IP returns False.

• Defects 5 and 6 – find_signature:

- range(n - m) excludes the last possible starting position. Use range(n - m + 1).
- j == m - 1 can be true even when the final character mismatches and the loop breaks. Use a matched flag or for-else.
- For "FAILFAIXFAIL" and "FAIL", the valid positions are 0 and 8.
- Boundary tests: a pattern at the end is found; a longer pattern returns []; an empty pattern is handled explicitly.

• **Defects 7, 8 and 9 – closest_logins:**
- Start min_dist at infinity so the first valid pair can be selected.
- Start j at i + 1 to avoid comparing an event with itself and checking each pair twice.
- Update when d < min_dist because the required pair is the closest.
- For fewer than two events, return (None, None).

• **Verification:**
- Run each function after its correction using the supplied data.
- Also test empty blocklist, absent IP, end-position signature, pattern longer than log, equal severities and fewer than two login events.

(c) Optimise the Corrected Module (10 Marks): Without changing any output, improve the corrected module for the production workload. Justify whether insertion sort remains suitable for nearly sorted alerts; propose a better approach for repeated blocklist lookups; make sure the signature scan performs no redundant comparisons; and, for closest_logins, eliminate self and repeated pairs, remove unnecessary square-root computations and prune pairs that cannot be closest.

ANSWER:

• Alerts:
- Insertion sort is suitable when alerts are nearly sorted because it needs few shifts and has $\Theta(n)$ best-case time.
- For 50,000 alerts, Python's built-in stable sort is generally faster and has O(n log n) time. Use key=lambda alert: alert[1], reverse=True. Python sorting is stable, so equal severities retain arrival order.

• Blocklist:
- Convert the blocklist to a set once and use ip in block_set.
- Set lookup is O(1) on average, compared with O(b) for a list scan, where b is the blocklist size.

• Signature matching:
- Use a standard brute-force scan with range(n - m + 1).
- Stop comparing a position as soon as a character differs.
- Record a position only when all m characters match. This avoids redundant comparisons and false matches.

• Closest login locations:
- Check only pairs where j > i, so self-pairs and duplicate pair checks are removed.
- Compare squared distances (dx*dx + dy*dy); calculate the square root only once for the final answer.

- For a larger batch, use a divide-and-conquer closest-pair algorithm: sort by x and y, solve the two halves, and check only points in the narrow middle strip. This implementation sorts the strip at each recursive level, so it takes $O(n \log^2 n)$ time, still better than $O(n^2)$.
- Keep the coordinates in the same unit (the question states kilometres).

(d) Analyse Before and After (10 Marks): Derive the number of basic operations and the time complexity of each function before and after optimisation. Measure the distance computations of the brute-force and optimised closest_logins for at least three batch sizes between 1,000 and 8,000 events, and compare insertion-sort comparisons on nearly sorted and random alerts. Explain whether the measurements agree with your analysis.

ANSWER:

• Before optimisation:
- M1 insertion sort: best $\Theta(n)$, average/worst $\Theta(n^2)$; comparisons and shifts can be quadratic.
- M2 list-based blocklist search: $O(b)$ per lookup; for q queries, $O(qb)$.
- M3 brute-force string matching: at most $(n - m + 1)m$ character comparisons, so $O(nm)$.
- M4 brute-force closest pair: $n(n - 1)/2$ distinct-pair distance calculations, so $\Theta(n^2)$.

• After optimisation:
- M1 built-in stable sort: $O(n \log n)$ worst-case.
- M2 set lookup: $O(1)$ average per query; $O(b)$ to build the set.
- M3 corrected brute-force scan: $O((n - m + 1)m)$, with no skipped final position or false match.
- M4 divide-and-conquer closest pair: $O(n \log n)$.

• Closest-pair brute-force distance counts:
- n = 1,000: 1,000 × 999 / 2 = 499,500.
- n = 4,000: 4,000 × 3,999 / 2 = 7,998,000.
- n = 8,000: 8,000 × 7,999 / 2 = 31,996,000.
- These are exact counts for the distinct-pair brute-force method. The divide-and-conquer count depends on the data distribution; measure it with the supplied test harness rather than assuming a fixed count.

• Insertion-sort comparisons:
- Nearly sorted input: close to $n - 1$ comparisons, so growth is close to linear.
- Random input: average comparisons and shifts grow quadratically, $\Theta(n^2)$.
- The observations agree with the complexity analysis: insertion sort works well for nearly sorted data but is not the best choice for large random batches.

(e) Deliver Production-Quality Code (8 Marks): Submit the final, optimised module with docstrings, meaningful comments and a test harness that checks every function, including the boundary cases from (b). Commit the code to your GitHub repository and share the link through Google Classroom.

ANSWER:

• The following corrected module uses a stable sort, set-based blocklist lookup, complete brute-force signature matching and a divide-and-conquer closest-pair method.
• It includes tests for the supplied data and the required boundary cases.
• Run the code in Python 3. The test harness prints the measured closest-pair distance-computation counts for 1,000, 4,000 and 8,000 generated events.

```python
import math
import random


def sort_alerts(alerts):
    """Return alerts sorted by descending severity; preserve equal-severity order."""
    return sorted(alerts, key=lambda alert: alert[1], reverse=True)


def is_blocked(blocklist, ip):
    """Return True when the IP address is in the blocklist."""
    return ip in set(blocklist)


def find_signature(log, pattern):
    """Return every starting index where pattern occurs in log."""
    if pattern == "":
        return list(range(len(log) + 1))

    n, m = len(log), len(pattern)
    hits = []

    if m > n:
        return hits

    for i in range(n - m + 1):
        j = 0
        while j < m and log[i + j] == pattern[j]:
            j += 1
```

```
        if j == m:
            hits.append(i)

    return hits


def closest_logins(events):
    """Return the closest pair of points and their Euclidean distance."""
    if len(events) < 2:
        return None, None

    points = [(x, y, i) for i, (x, y) in enumerate(events)]
    points.sort(key=lambda p: (p[0], p[1]))

    def distance_squared(a, b):
        dx = a[0] - b[0]
        dy = a[1] - b[1]
        return dx * dx + dy * dy

    def solve(px):
        size = len(px)

        if size <= 3:
            best = float("inf")
            best_pair = None
            for i in range(size):
                for j in range(i + 1, size):
                    d2 = distance_squared(px[i], px[j])
                    if d2 < best:
                        best = d2
                        best_pair = (px[i], px[j])
            return best, best_pair

        mid = size // 2
        mid_x = px[mid][0]

        left_best, left_pair = solve(px[:mid])
        right_best, right_pair = solve(px[mid:])

        if left_best <= right_best:
            best, best_pair = left_best, left_pair
        else:
            best, best_pair = right_best, right_pair
```

```
        strip = [p for p in px if (p[0] - mid_x) ** 2 < best]
        strip.sort(key=lambda p: p[1])

        for i in range(len(strip)):
            j = i + 1
            while j < len(strip) and (strip[j][1] - strip[i][1]) ** 2 < best:
                d2 = distance_squared(strip[i], strip[j])
                if d2 < best:
                    best = d2
                    best_pair = (strip[i], strip[j])
                j += 1

        return best, best_pair

    best_sq, pair = solve(points)
    result_pair = (pair[0][2], pair[1][2])
    return result_pair, math.sqrt(best_sq)


def test_module():
    alerts = [("A1", 3), ("A2", 9), ("A3", 5),
              ("A4", 9), ("A5", 1)]
    expected_alerts = [("A2", 9), ("A4", 9), ("A3", 5),
                       ("A1", 3), ("A5", 1)]
    assert sort_alerts(alerts) == expected_alerts

    blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
    assert is_blocked(blocklist, "10.0.0.9") is True
    assert is_blocked(blocklist, "8.8.8.8") is False
    assert is_blocked([], "8.8.8.8") is False

    assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]
    assert find_signature("ABCEND", "END") == [3]
    assert find_signature("ABC", "ABCDE") == []
    assert find_signature("ABC", "") == [0, 1, 2, 3]

    events = [(2, 3), (12, 30), (40, 50),
              (5, 1), (12, 10), (3, 4)]
    pair, distance = closest_logins(events)
    assert set(pair) == {0, 5}
    assert abs(distance - math.sqrt(2)) < 1e-9
    assert closest_logins([]) == (None, None)
```

```
    assert closest_logins([(1, 2)]) == (None, None)

    print("All tests passed.")


def benchmark_closest_pair():
    random.seed(7)
    for n in (1000, 4000, 8000):
        events = [(random.random() * 10000,
                   random.random() * 10000) for _ in range(n)]
        pair, distance = closest_logins(events)
        brute_force_count = n * (n - 1) // 2
        print("Events:", n)
        print("Brute-force distance calculations:", brute_force_count)
        print("Optimised result distance:", distance)
if __name__ == "__main__":
    test_module()
    benchmark_closest_pair()
```

• Note: The code returns the closest pair's original event indices. The divide-and-conquer method avoids checking every possible pair. The benchmark reports the exact brute-force count and the optimised result; actual timings depend on the computer and input data.

• **GitHub submission:**
- Save the file as sentinelops.py.
- Run it and confirm that all tests pass.
- Upload the file to your GitHub repository.
- Copy your repository link and submit it through Google Classroom.

• Conclusion: The corrected module sorts alerts properly, identifies blocked IPs, finds all signature occurrences and selects the nearest login locations. The optimisations make it more suitable for the SOC workload.

Rubrics – Debugging and Optimisation Task

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Code Quality	15	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20				
Debugging	25				
Optimization	20				
Analysis	20				
Code Quality	15				
Total Marks					

Code of Conduct

I __ **DHARSHITHA.K(192521093)** __ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____